

Word Segmentation and Tokenization

From Rule-Based Methods to the Age of Large Language Models

Dr. Ye Kyaw Thu

Lab Leader, Language Understanding Lab, Myanmar
Visiting Professor, Core-AI Lab, NECTEC, Thailand

`yktnlp@gmail.com`

`https://sites.google.com/site/yekyawthunlp/`

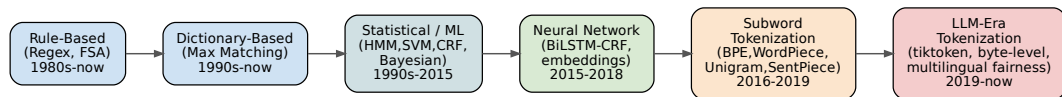
July 12, 2026

- 1 Why Word Segmentation Matters
- 2 Segmentation Challenges Across Languages
- 3 Rule-Based Segmentation
- 4 Dictionary-Based Segmentation & Maximum Matching
- 5 Statistical & Machine-Learning Approaches
- 6 Neural Network-Based Segmentation
- 7 The Subword Tokenization Era
- 8 Multilingual Tokenization in the LLM Era
- 9 Segmentation and Semantic / Vector Representation
- 10 Summary and R&D Trends
- 11 References

By the end of this lecture, you should be able to:

- Explain **why** word segmentation/tokenization matters for classic NLP *and* LLMs
- Describe the segmentation **challenges** specific to Chinese, Japanese, Thai, and Myanmar
- Trace the **history**: rule-based → dictionary → statistical/ML → neural → subword → LLM tokenization
- Explain **Maximum Matching, HMM, SVM, CRF**, and neural sequence-labeling approaches
- Explain **BPE, WordPiece, Unigram LM, SentencePiece**, and byte-level tokenization
- Discuss **multilingual tokenizer fairness** and its cost/latency implications
- Understand the relationship between segmentation granularity and **semantic representation**
- Know the key Myanmar-language segmentation tools: **sylbreak, sylbreak4all, myWord, oppaWord, NgaPi**

Roadmap for Today

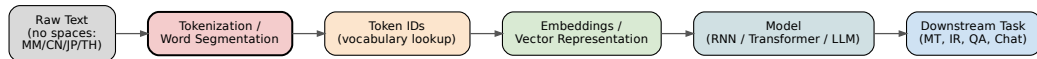


Six eras of word segmentation research – we will walk through each, with real papers, algorithms, and my own group's tools as running case studies.

What Is Word Segmentation / Tokenization?

- **Word segmentation**: dividing a stream of characters into meaningful units (syllables, words, morphemes)
- **Tokenization** (modern/LLM usage): converting raw text into a sequence of *tokens* – units a model actually consumes (may be words, subwords, characters, or bytes)
- In English, whitespace gives a first approximation "for free" – but even then it is not solved (see later slide)
- In **Chinese, Japanese, Thai, Myanmar (and many other Asian scripts)**, there is **no whitespace between words at all** – segmentation must be inferred
- Tokenization is the **very first step** of almost every NLP/LLM pipeline – errors here propagate to everything downstream

Where Tokenization Sits in the Pipeline



Whatever a model “sees” is entirely determined by this first step. A bad split here cannot be fixed later by a bigger model.

- **Machine Translation:** word alignment, phrase tables, and modern NMT all depend on consistent input units
- **POS tagging / Parsing:** tag sets and grammars are defined over word/morpheme units
- **Information Retrieval:** indexing and query matching require consistent term boundaries
- **Text-to-Speech / G2P (grapheme-to-phoneme):** syllable boundaries directly affect pronunciation rules
- **Romanization / Transliteration:** needs a defined unit (syllable is often the natural unit for Myanmar)
- Segmentation choice changes **vocabulary size**, **data sparsity**, and **OOV (out-of-vocabulary) rate**

- Neural models – including LLMs – do not read characters or words directly; they read **token IDs** mapped from a fixed **vocabulary**
- The tokenizer is typically **frozen at pretraining time** – you cannot change it later without retraining embeddings
- Tokenization determines:
 - How many tokens a given text costs → **API price** (billed per token)
 - How much text fits in the **context window** (a fixed number of tokens)
 - Inference **latency** (more tokens = more forward passes)
 - How well the model can represent **rare words, numbers, code, and non-English scripts**
- A well-known real-world consequence: some languages need **far more tokens** than English for the same content – more on this later

A Concrete Illustration: Token Cost Differs by Segmentation

Consider the Myanmar sentence meaning “Forget me”: ခွင့်လွှတ်ပါ။

Unit	Segmentation	# Units
Character	ခ ဝ င် ဝ လ ဝ ာ တ ဝ ... (individual Unicode code points)	~12+
Syllable (sylbreak)	ခွင့် လွှတ် ပါ ။	4
Word (myWord)	ခွင့်လွှတ် ပါ ။	3

Same sentence, same meaning – but a 3–4× difference in unit count depending on the segmentation strategy chosen. This directly affects model training data statistics *and*, for LLM APIs, your bill.

- Tokenization is not a “preprocessing footnote” – it shapes what a model can learn and how much it costs to use
- For space-delimited languages the problem is easy to ignore (but not actually solved)
- For Chinese/Japanese/Thai/Myanmar-family scripts, segmentation is a genuine, unavoidable research problem
- Next: why is this so hard across different languages?

The Core Problem: Not All Scripts Use Spaces

- English, French, German, ...: words are (mostly) space-delimited
- **Chinese, Japanese, Thai, Khmer, Lao, Myanmar: no spaces between words** in normal writing
- Some of these languages use spaces only for *phrase*-level breaks (Myanmar), not word-level breaks
- This means: for roughly a quarter of the world's population, “just split on whitespace” is not an option at all

Even English Is Not Actually Trivial

- Contractions: *don't* → *do* + *n't*? or one token?
- Compounds: *New York*, *ice cream*, *state-of-the-art* – are these one word or several?
- Hyphenation, clitics (*'s*), abbreviations (*U.S.A.*), numbers (*3.14*, *\$1,000.50*)
- Emoji, URLs, hashtags, code snippets in modern text
- Punctuation attachment: *"Hello!"* vs *Hello* + *!*

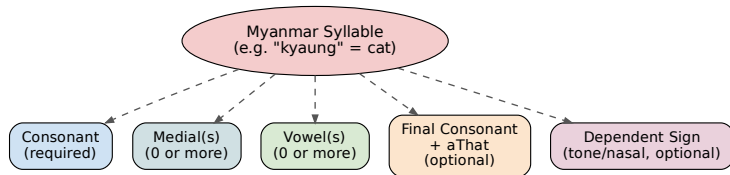
This is exactly why myWord's own experiments needed an explicit **NPMI phrase-detection step** to decide when a bigram like *New_York* should be treated as one unit – even in English. (See Word2Vec, Mikolov et al., 2013.)

- Written Chinese has no word delimiters; a sentence is a sequence of characters (hanzi)
- The same character sequence can often be split into different words depending on context – ambiguous segmentation is the norm, not the exception
- Segmentation standards differ across corpora (PKU, MSR, CityU) – there is no single “correct” Chinese word segmentation
- **Key empirical finding:** for some NLP tasks (notably neural machine translation), **character-level** segmentation of Chinese can perform competitively with, or better than, word-level segmentation – avoiding segmentation errors altogether by not segmenting into words at all

- Japanese text mixes **Kanji** (logographic), **Hiragana** and **Katakana** (syllabic), plus Latin letters and digits – all with no spaces
- Word segmentation quickly becomes tangled with **morphological analysis** (verb conjugations, particles)
- Standard tools: **MeCab**, **JUMAN++**, **Sudachi** – typically dictionary + statistical (CRF) hybrids
- Early influential work: HMM-based Japanese segmentation (Papageorgiou, 1994); SVM-based segmentation (Kudo & Matsumoto, 2001)

- Thai script has no spaces between words (only between some phrases/sentences)
- Classic approach: dictionary + longest matching, similar to early Chinese/Myanmar approaches
- Modern tools: **deepcut** (CNN-based), **attacut** (fast neural segmenter), **PyThaiNLP**
- Same fundamental tension as Chinese/Myanmar: dictionary coverage vs. statistical generalization to unseen words

Myanmar (Burmese): A Syllabic Abugida Script



A Myanmar syllable = consonant (required) + medial(s) + vowel(s) + optional final consonant/aThat + optional dependent sign. Vowels/medials are stored *after* the consonant in Unicode logical order even though they render before/around it visually.

- **No reliable word spacing:** spaces are used loosely for phrase readability, not word boundaries, and are often simply omitted
- A word may be 1 to 4+ syllables long, with no closed-class marker of where a word ends
- Ambiguous stacking: consonant + subscript symbol (virama, ့) vs. consonant + aThat (်) look similar but have different syllable-boundary implications (e.g. တက္ကသိုလ် “university”)
- Myanmar script is also used, with adaptations, to write **eight other ethnic languages** – each with its own consonant/medial/tone inventory

Nine Major Ethnic Languages of Myanmar (syllbreak4all)

Language	Family / Notes	Script Notes
Burmese (Myanmar)	Sino-Tibetan, official language, tonal	Full inventory: 34 consonants, 4 medials, 8 vowels
Shan	Tai-Kadai, tonal (5 tones + emphasis)	19 consonants, distinct Shan digits
Pa'O	Central Karenic	Uses Burmese script + phonetic marks (Mine Ngar)
Pwo Karen / S'gaw Karen	Karenic	Burmese-derived alphabets, tone marks, no subscript/aThat
Rakhine (Arakanese)	Tibeto-Burman	Nearly identical script to Burmese
Myeik, Dawei (Tavoyan)	Burmese dialects	Burmese script, distinct phonology/tone contours
Mon	Austro-Asiatic (Mon-Khmer)	More superscript consonants than Burmese, non-tonal

Source: Thu et al., *syllbreak4all*, iSAI-NLP 2021.

- Ding, Thu, Utiyama & Sumita (2016) and Thu et al. (2013a) systematically compared segmentation granularities for Myanmar Statistical MT: character, syllable, word (unsupervised/semi-supervised/dictionary-based)
- **Finding:** the highest-quality MT was attained *either* with simple **syllable sequences** (no word segmentation at all) *or* with **Maximum Matching + a monolingual dictionary** – more complex unsupervised/semi-supervised word segmenters did *not* outperform these simpler baselines
- Parallels the Chinese finding: sometimes a **finer, more robust unit** (syllable/character) beats an imperfect word segmenter, because segmentation errors are themselves a source of noise for downstream models
- **Takeaway for engineers:** always benchmark segmentation granularity empirically for your task – there is no universally “correct” choice

- Roughly a quarter of the world writes without word spaces – segmentation is a first-class NLP problem, not an edge case
- Chinese, Japanese, Thai, and the Myanmar language family each have distinct challenges (ambiguity, mixed scripts, tone/medial complexity)
- Empirically, coarser units (syllable/character) sometimes outperform imperfect word segmentation for downstream tasks like MT
- Next: how have researchers actually solved (or worked around) this problem, historically?

- Encode linguistic knowledge directly as **rules** – regular expressions, finite-state automata, hand-written grammars
- Advantages: no training data needed, fast, fully interpretable, easy to debug and extend
- Disadvantages: brittle to exceptions, requires a linguist/engineer to encode every rule, doesn't generalize automatically to new vocabulary
- For syllable-structured scripts (Myanmar, and to some extent Chinese/Japanese/Thai), the syllable itself has a well-defined, rule-describable internal grammar – making regex/FSA approaches surprisingly effective at the **syllable** level

My own tool, **sylbreak** (2014), segments Myanmar text into syllables using a *single line* of Perl/Python regular expression:

```
BreakPattern = re.compile(  
    r"((?<!" + ssSymbol + r")["+myConsonant+r"] "  
    r"(?!["+aThat+ssSymbol+r"])"  
    r"|["+enChar+otherChar+r"])", re.UNICODE)
```

Rule in plain English: place a break before a consonant, unless (a) that consonant is preceded by a subscript-stacking symbol, or (b) it is itself followed by an aThat/subscript symbol. Also break before any English letter/digit/other-char class. GitHub: <https://github.com/ye-kyaw-thu/sylbreak>

Case Study: sylbreak – One Line of Regex

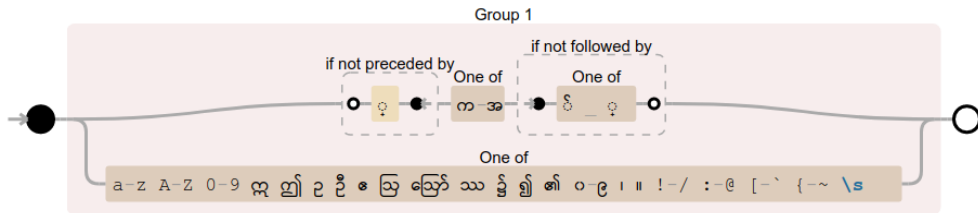


Fig. Visualization of the sylbreak regular expression.

Why a Single Regex Line Works So Well Here

- Myanmar syllable structure is **highly regular** at the Unicode level: consonant → (medial) → (vowel) → (final/aThat) → (sign)
- The key insight: a **new syllable begins at a consonant**, *except* when that consonant is:
 - immediately followed by an a-That (◌်) – meaning it closes the *previous* syllable, or
 - immediately followed by a subscript-stacking symbol (◌_◌) – meaning it is stacked onto the previous consonant (common in Pali/Sanskrit loan words)
- This one rule handles the overwhelming majority of real Myanmar text correctly, with no training data required at all
- Known limitation (by design choice): stacked consonant clusters like တက္ကသိုလ် (“university”) are broken as တက္က + သိုလ် rather than the linguistically “pure” 3-syllable form – a deliberate simplification that works better for downstream NLP tasks

- **sylbreak4all** (Thu et al., iSAI-NLP 2021) generalizes the sylbreak regex approach to **nine major languages/dialects** written in Myanmar-derived scripts: Burmese, Shan, Pa'O, Pwo Karen, S'gaw Karen, Rakhine, Myeik, Dawei, Mon
- Each language gets its **own tailored RE pattern**, changing the consonant/vowel/other-char character classes to match that language's Unicode inventory
- Burmese-script pattern reused directly for Dawei, Myeik, Rakhine, and Pa'O (they share the same underlying script)
- Shan needs a modified pattern (no subscript symbol exists in Shan)
- Karen languages (Pwo/S'gaw) use only the Myanmar consonant set (no aThat/subscript complexity)

sylbreak4all: Regular Expressions for Syllable Breaking of Nine Major Ethnic Languages of Myanmar

1 st Ye Kyaw Thu <i>NECTEC, Thailand</i> yktnlp@gmail.com	2 nd Hlaing Myat Nwe <i>UTYCC, Myanmar</i> hlaingmyatnwe@utycc.edu.mm	3 rd Hnin Aye Thant <i>UTYCC, Myanmar</i> hninayethant@utycc.edu.mm	4 th Hay Man Htun <i>UTYCC, Myanmar</i> haymanhtun@utycc.edu.mm
4 th Htay Mon <i>Sakhangyi Middle School</i> hmon685@gmail.com	4 th May Myat Myat Khaing <i>UTYCC, Myanmar</i> maymyatmyatkhaing@utycc.edu.mm	4 th Hsu Pan Oo <i>UCS (Thaton), Myanmar</i> mihsupano@gmail.com	4 th Pale Phyu <i>UCS (Thaton), Myanmar</i> palephyu12345@gmail.com
4 th Nang Aeindray Kyaw <i>UTYCC, Myanmar</i> nangaendraykyaw@utycc.edu.mm	4 th Thazin Myint Oo <i>UCSY, Myanmar</i> queenofthazin@gmail.com	4 th Thazin Oo <i>UCS (Thaton), Myanmar</i> thazinoo@ucstt.edu.mm	4 th Thet Thet Zin <i>UCS (Thaton), Myanmar</i> thetthetzin121@gmail.com
4 th Thida Oo <i>UCS (Thaton), Myanmar</i> thidaroo744@gmail.com			

Abstract—Unlike many other western languages, the Myanmar language uses a syllabic writing system and no space between words. Syllable segmentation is the necessary preprocess for natural language processing (NLP) tasks such as grapheme-to-phoneme (g2p) conversion, machine translation, romanization, and so on. In this study, sylbreak4all, a syllable segmentation tool, was developed for nine major ethnic languages of Myanmar, and they are Burmese, Shan, Pa'o, Pwo Kayin, S'gaw Kayin, Rakhine, Myeik, Dawei, and Mon by using regular expression (RE) patterns.

Index Terms—sylbreak4all, Syllable breaking, Syllable segmentation, Regular Expression

I. INTRODUCTION

Myanmar language (Burmese) is the official language

with Unicode such as Myanmar3, Padauk, and so on. These languages are Burmese, Shan, Pa'o, Pwo Kayin, S'gaw Kayin, Rakhine, Myeik, Dawei, and Mon. Non-Myanmar characters, such as English and some special characters, are considered in this study. The Unicode block for Myanmar text is U+1000 to U+109F that covers all ethnic languages of Myanmar. In this study, we also used U+AA7B for Pa'o tone Mine Ngar (ၵ) from Myanmar Unicode standard extended version A [1].

II. NINE ETHNIC LANGUAGES OF MYANMAR

There are around 100 languages and dialects used in Myanmar. They represent the four major language families as Sino-Tibetan, Austro-Asiatic, Tai-Kadai, and Indo-

Language	Unsegmented	Segmented
Burmese	ခွင့်လွှတ်ပါ။	ခွင့် လွှတ် ပါ။
Shan	ပွဲ,ခွာင်းငလး။	ပွဲ, ခွာင်း ငလး။
S'gaw Karen	ဝံသးစူအီၤ.	ဝံ သး စူ အီၤ.
Pwo Karen	ဖျါခွံယးဆံး.	ဖျါ ခွံ ယး ဆံး.
Pa'O	ခွင်လွတ်ဟွင်း	ခွင် လွတ် ဟွင်း
Mon	သုးအခါင်ညီ	သုး အ ခါင် ညီ

Source: Thu et al., *sylbreak4all*, Table VI, iSAI-NLP 2021.

- Evaluated on **139,136 sentences**, totaling **1,910,822 syllables** across all nine languages
- Manual check on 1,000 randomly-sampled sentences: **100% Precision, 100% Recall, 100% F-measure** for correctly-typed Unicode input
- Only Mon showed a small residual error rate ($\sim 0.1\%$), concentrated in rare medial-stacking cases (*byaee swa/ ù* and *w* combined with *aThat*)

Language	Sentences	Syllables	Unique Syllables
Burmese	11,001	381,335	1,903
S'gaw Karen	68,571	890,731	1,693
Shan	18,338	182,319	1,318
Mon	4,160	19,812	1,320

Rule-Based Segmentation: Strengths and Limits

- + Zero training data required; deterministic and fully explainable
- + Extremely fast (single regex pass); trivial to deploy anywhere (Perl, Python, Java, JS, C++ – sylbreak ships in all of these)
- + Excellent for well-structured units like **syllables**, where the “grammar” is genuinely simple and closed-form
 - Does not scale to **word**-level segmentation, where boundaries depend on vocabulary and context, not just character-class patterns
 - Any exception requires a human to notice it and hand-edit the rule
 - Natural next step: combine rules with a **dictionary** to move from syllables to words

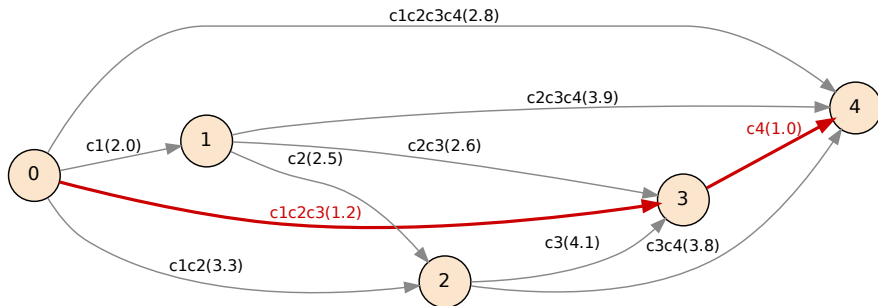
- Idea: maintain a **lexicon** of known words; scan the input and match against it
- Simple, effective, and historically the dominant approach for Chinese, Japanese, Thai, and Myanmar before statistical methods matured
- Two classic decision strategies:
 - **Forward Maximum Matching (FMM)**: from the current position, take the *longest* dictionary word that matches, then advance
 - **Backward Maximum Matching (BMM)**: same idea, scanning right-to-left
 - **Bidirectional Maximum Matching (Bi-MM)**: run both, and use agreement / heuristics to resolve conflicts
- Myanmar-specific applications: Thet et al. (2008, bigram-based boundary decisions), Htay & Murthy (2008, word-list from monolingual corpus) – both operate as *syllable-break followed by Maximum Matching*

- + Simple, fast, no training required beyond building the dictionary
- + High **recall** in practice – when in doubt, it still produces *some* segmentation (often defaulting to syllables when no word matches)
- Precision suffers wherever the dictionary lacks coverage: **Out-Of-Vocabulary (OOV)** words, named entities, neologisms
- Greedy longest-match can be *wrong*: e.g. English “the therapist” greedily matched could wrongly prefer “the rapist” + “ist” style errors in some languages
- This tension – **dictionary coverage vs. generalization to unseen words** – is the central theme of the next two sections

- My own tool **myWord** (2021) segments Myanmar text at **four levels**: syllable, sub-word, word, and phrase
- Word-level segmentation uses **unigram + bigram dictionaries**, built from a manually word-segmented corpus of **12+ million words**
- Decoding uses the **Viterbi algorithm** (Viterbi, 1967) – the same dynamic-programming algorithm used in HMM decoding, ASR, and DNA sequencing
- Key idea: treat segmentation as a **shortest-path problem** over a graph of candidate syllable groupings, where edge weight = negative log-probability from the n-gram dictionary
- GitHub: <https://github.com/ye-kyaw-thu/myWord>

Viterbi Decoding: Worked Example

Input syllables: **c1 c2 c3 c4** (Myanmar example: “ဆ ရ က က”, i.e. “sa-ya-ka” = “I am a teacher”-ish fragment)



Each edge = a candidate segment with its $-\log P$ cost from the unigram/bigram dictionary. Viterbi finds the **lowest total-cost path** in $O(T \cdot S^2)$ instead of brute-force $O(S^T)$. Here the best path (red) segments as **c1c2c3 | c4**, i.e. “ဆရာ | က”.

- **Closed test** (1,000 sentences from training data): Word F-measure = **88.14%**, Boundary Accuracy = **97.50%**
- **Open test** (5,626 unseen sentences, 135K+ words): Word F-measure = **88.08%**, Boundary Accuracy = **97.60%**
- Very close closed/open performance – suggests the n-gram dictionary generalizes reasonably well beyond its training data
- myWord also supports **phrase segmentation** via **NPMI** (Normalized Pointwise Mutual Information, Bouma 2009) – automatically discovering collocations like ကျွန်တော်_က without any labeled phrase data, directly inspired by the Word2Vec phrase-detection idea (Mikolov et al., 2013)

- Key idea (widely used for Chinese/Japanese): treat word segmentation as **character-level classification**
- Standard tagging scheme – **BIES** (or BMES): each character is tagged
 - **B** – Begin of a word
 - **I/M** – Inside/Middle of a word
 - **E** – End of a word
 - **S** – Single-character word
- This turns segmentation into a standard **supervised sequence labeling** problem – exactly like POS tagging or NER – opening the door to HMM, SVM, CRF, and later neural sequence models

- Model characters (observed states) and BIES tags (hidden states) with a **Hidden Markov Model**
- Early example: Papageorgiou (1994), HMM-based Japanese word segmentation
- Decoding again uses the **Viterbi algorithm** – the same dynamic-programming machinery we just saw for myWord's dictionary-based decoding
- Limitation: HMM's Markov (local) transition assumption struggles to capture longer-range or overlapping features (e.g. character type, dictionary membership, position-in-word) simultaneously

- **Support Vector Machines** classify each character-boundary as “split” or “no split” using rich, overlapping local features (surrounding characters, character type, dictionary lookups)
- Kudo & Matsumoto (2001): SVM-based Japanese word segmentation and text chunking
- Xue (2003) and related work: character-tagging approaches to Chinese word segmentation using maximum-entropy/SVM classifiers
- Advantage over HMM: can use arbitrary, overlapping features per decision point – not limited to a Markov chain
- Disadvantage: classifies each boundary somewhat independently – no built-in global sequence consistency (this is exactly what CRFs fix next)

- **Conditional Random Fields** (Lafferty et al., 2001) model the *whole tag sequence* jointly, avoiding the “label bias” problems of simpler classifiers
- Peng, Feng & McCallum (2004): CRFs for Chinese word segmentation, treating it as BIES sequence labeling – became a dominant paradigm through the mid-2000s–2010s
- Tseng et al. (2005): a CRF-based Chinese segmenter at the SIGHAN bakeoff, highly influential
- Applied to Myanmar too: Zhang, Mao, Yu, Wang, Li & Zhang (2019), “Word Segmentation for Burmese Based on Dual-Layer CRFs”
- CRFs became, for roughly a decade, the **de facto standard** for supervised Chinese/Japanese/Myanmar word segmentation before neural approaches took over

- What if we have **no** labeled segmentation data at all – only raw text?
- Goldwater, Griffiths & Johnson (2009): a **non-parametric Bayesian** word segmenter using a **Dirichlet Process (DP)** prior
- Core idea: model text generation as repeatedly drawing “segments” from a distribution G , where G itself is drawn from $DP(\alpha, G_0)$

$$G \mid \alpha, G_0 \sim DP(\alpha, G_0), \quad s_k \mid G \sim G$$

- G_0 (the **base measure**) controls how novel, never-seen segments get generated – typically a simple length-based (Poisson) + character-based (uniform) spelling model
- Inference via **Gibbs sampling** / **forward-filtering backward-sampling (FFBS)** – reusing dynamic programming ideas again
- Fully unsupervised, but tends to have lower precision – it discovers frequently co-occurring character sequences, which are not always real words

- Thu, Finch, Sumita & Sagisaka, *Integrating Dictionaries into an Unsupervised Model for Myanmar Word Segmentation*, **COLING 2014 Workshop on South and Southeast Asian NLP**
- Question: can we keep the **coverage advantage** of unsupervised Bayesian segmentation while injecting the **precision** of a dictionary?
- Proposed **three augmented models**, all built on top of the Goldwater et al. (2009) Bayesian baseline:
 - ① **Dictionary Augmented Model** – the generative process can “choose” to emit a word directly from a dictionary instead of the unsupervised sub-model
 - ② **Dictionary Set Augmented Model** – splits the dictionary into POS-labeled subsets, each competing with its own learned probability
 - ③ **Language Model Augmented Model** – trains an n-gram LM on syllable-segmented dictionary words, replacing the naive uniform base measure G_0
- Link: <https://aclanthology.org/W14-5503/>

Integrating Dictionaries into an Unsupervised Model for Myanmar Word Segmentation

Ye Kyaw Thu

NICT

Keihanna Science City

Kyoto, Japan

yekyawthu@nict.go.jp

Andrew Finch

NICT

Keihanna Science City

Kyoto, Japan

andrew.finch@nict.go.jp

Eiichiro Sumita

NICT

Keihanna Science City

Kyoto, Japan

eiichiro.sumita@nict.go.jp

Yoshinori Sagisaka

GITI/Speech Science Research Lab.

Waseda University

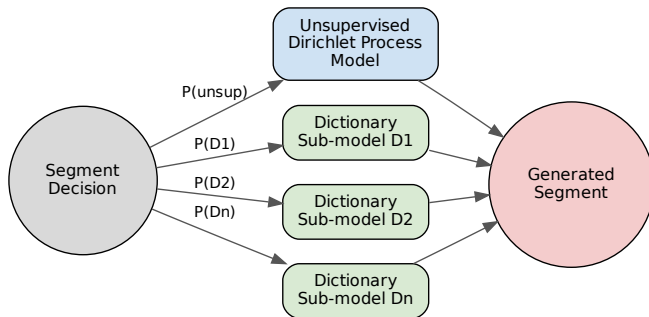
Tokyo, Japan

ysagisaka@gmail.com

Abstract

This paper addresses the problem of word segmentation for low resource languages, with the main focus being on Myanmar language. In our proposed method, we focus on exploiting limited amounts of dictionary resource, in an attempt to improve the segmentation quality of an unsupervised word segmenter. Three models are proposed. In the first, a set of dictionaries (separate dictionaries for different classes of words) are directly introduced into the generative model. In the second, a language model was built from the dictionaries, and the n-gram model was inserted into the generative model. This model was expected to model words that did not occur in the training data. The third model was a combination of the previous two models. We evaluated our approach on a corpus of manually annotated data. Our results show that the proposed methods are able to improve over a fully unsupervised baseline system. The best of our systems improved the F-score from 0.48 to 0.66. In addition to segmenting the data, one proposed method is also able to partially label the segmented corpus with POS tags. We found that these labels were approximately 66% accurate.

The Generative Process (Redrawn)



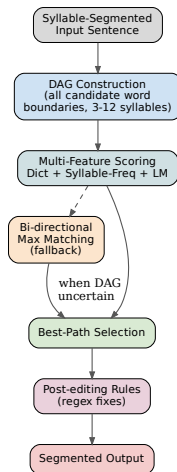
At each generation step, the model *competes* between the fully unsupervised Dirichlet Process sub-model and one or more dictionary sub-models, each with a learned mixing probability—re-estimated after every training iteration. (Redrawn concept from Thu et al., COLING 2014, Fig. 1.)

Results: Dictionary Integration Helps a Lot

Method	Precision	Recall	F-score
Unsupervised (baseline)	82.27	33.82	0.48
Maximum Matching	78.39	99.42	0.88
Dictionary Augmented	89.67	57.34	0.70
Dictionary Set Augmented	89.46	51.99	0.66
Language Model (LM) Augmented	88.15	31.10	0.46
Dictionary Set + LM	91.27	49.76	0.64

- Best system improved F-score from **0.48** → **0.66** over the pure unsupervised baseline
- Maximum Matching still had the single highest F-score (0.88) here – but at the cost of needing a good dictionary; the point of this work was improving the *unsupervised* family, not beating MM outright
- LM-only augmentation underperformed alone, but **helped precision** when combined with the dictionary set model
- One proposed method could also **partially POS-label** the segmented output (up to 65.7% labeling accuracy) as a free byproduct

- My newer tool **oppaWord** (2025)—“Super Fast Myanmar Word Segmenter”—combines several of the ideas above into one hybrid, **training-free** pipeline.
- Combines:
 - **DAG construction** over candidate segmentations
 - **Bidirectional Maximum Matching** as a fallback
 - **ARPA-format n-gram language model** for scoring
- GitHub: <https://github.com/ye-kyaw-thu/oppaWord>



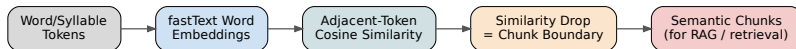
- **Speed**: segments the entire myPOS corpus (43,196 sentences) in under **3 seconds** – currently the fastest known Myanmar word segmenter, and faster than any neural segmenter
- **Accuracy** (with dictionary + Bi-MM fallback + post-editing rules, no LM): Word F1 = **0.90**, Vocabulary F1 = **0.91**
- **No training required** – just a dictionary, optional syllable-frequency table, and optional n-gram LM
- Ships with a comprehensive **error-analysis tool** categorizing over-segmentation, under-segmentation, and complex boundary errors – e.g. frequent over-segmentation of ကျန်တော် “I” into ကျန် + တော်
- Illustrates a broader lesson: **well-engineered classical methods can still beat neural models on speed**, and remain highly competitive on accuracy, for well-understood problems

- Replace hand-crafted CRF features with **learned character embeddings** fed into a **BiLSTM**, then decode the BIES tag sequence with a **CRF layer** on top – the “BiLSTM-CRF” architecture
- Chen, Qiu, Zhu, Liu & Huang (2015), “Long Short-Term Memory Neural Networks for Chinese Word Segmentation” – one of the first strong neural results on this task
- Cai & Zhao (2016): gated recursive neural networks for Chinese word segmentation, learning to combine candidate segment representations directly
- Advantage over CRF: character embeddings capture **distributional similarity** automatically (learned, not hand-designed features); still keeps CRF's global sequence consistency

- What changed: feature engineering was replaced by **representation learning** – embeddings, not hand-crafted n-gram/dictionary features
- What stayed the same: it is still framed as **sequence labeling** (BIES), still needs a labeled training corpus, and Viterbi-style decoding is still used at the output layer
- Neural segmenters generally pushed Chinese/Japanese segmentation **F-scores higher** than CRF baselines, at the cost of needing more compute and (usually) more training data
- Important historical footnote: for **Chinese-English NMT**, an empirical study (Wang, Zhou, Zhang & Zong, 2017) found the choice of granularity (word vs. subword vs. character) still matters enormously – neural segmentation quality alone doesn't settle the “what unit should I use” question

- Once text is segmented into words, the natural next step is to represent each word as a **dense vector** – word2vec (Mikolov et al., 2013), fastText (Bojanowski et al., 2016)
- **fastText**'s key innovation: represent a word as the sum of its **character n-gram** embeddings, not a single opaque vector
- This makes fastText naturally robust to **morphology and OOV words** – exactly the problem that plagues dictionary-based segmenters
- This is the bridge to my own semantic-chunking experiment, **NgaPi**

- **NgaPi** (ငါးဝံ, 2024) asks a different question from classical segmentation: not “where does a *word* end?” but “**where does a coherent chunk of *meaning* end?**”
- Built on **fastText word embeddings** (Bojanowski et al., 2016; Joulin et al., 2016)
- Core idea: compute the **cosine similarity** between consecutive token embeddings; a sharp **drop** in similarity signals a natural chunk boundary
- Directly inspired by “The 5 Levels of Text Splitting for Retrieval” (Greg Kamradt) – i.e., motivated by practical **RAG (Retrieval-Augmented Generation)** chunking needs, not classical word segmentation
- GitHub: <https://github.com/ye-kyaw-thu/NgaPi> – the approach is language-agnostic and has been tested on Myanmar text, but applies to any language with trained word embeddings



This reframes “segmentation” one level up: from character/syllable/word boundaries to **semantic** boundaries – directly useful for LLM-era applications like RAG document chunking, where you want chunks that preserve topical coherence rather than just fixed token counts.

Test Run with ma_thein.txt

ma_thein.txt ဖိုင်ထဲမှာက အောက်မှာ မြင်ရတဲ့အတိုင်း word segmentation မလုပ်ထားဘူး။ ပြီးတော့ blank line တချို့လည်း ပါဝင်ပါတယ်။

```
(base) ye@1st-gpu-server-197:~/ye/exp/sem-chunk/4github$ cat ma_thein.txt
```

ရွှေတိဂုံဘုရား အနောက်ဘက်စောင်းတန်းမှာ မသိန်း ဆိုတဲ့ ပန်းသည်တယောက်ရှိတယ်။ ပန်းသည်မသိန်းပုံကို သိပ္ပံမောင်ဝ ရေးဖွဲ့ခဲ့ပုံက ...။

“မသိန်းသည် ပန်းသည်တို့အနက် အတော်ပင် ယဉ်စစ် ရှိ၏။ အပြောလည်း ကောင်း၊ သဘောလည်း ကောင်း၊ အပြုအပြင်ကလည်း ကောင်း၊ သူ့မှာ အကောင်းချည်း စုနေလေငေး အဲ့ဒီခေတ်က တက္ကသိုလ်ကျောင်းသားတွေဟာ စနေ၊ တနင်္ဂနွေနေ့တွေမှာ အုပ်စုလိုက် ဘုရားတက်ပြီး မသိန်းဆိုင်က ပန်းတွေဝယ်၊ မသိန်းဆိုင်ရှေ့က ခုံမှာထိုင်ပြီး စကားတွေ ဖောင့်တချို့ရက်တွေဆို ရှေ့ကအဖွဲ့ ဖောင်ဖွဲ့နေတုန်း နောက်တဖွဲ့ရောက်လာလို့ မထချင် ထချင်နဲ့ နေရာဖယ်ပေးရဆိုပဲ။ ဒီထဲ မောင်သိန်းခိုင်ဆိုတဲ့ ကျောင်းသားကတော့ တကိုယ်တော်။ သူက မသိန်း ဆိုင်သိမ်းချိန်အထိနေပြီး ဆိုင်သိမ်းပြီးရင် အတူပြန်တဲ့ထိ မသိန်းနဲ့ ရင်းနှီးတယ်။ ဒီလိုနဲ့ ကျောင်းသားဟောင်းတွေက ကျောင်းပြီးသွားလို့ နေရပ်အသိသီးဆီ ဘဝခရီးရုံ နောက် ကျောင်းသားအသစ်တွေ ရောက်လာပြီး မသိန်းဆိုင်မှာ ပန်းဝယ်ကြ၊ စကားတွေဖောင်ဖွဲ့ကြနဲ့။ ဆယ်စုနှစ် နှစ်ခုလောက်ရှိတော့ မသိန်းဆိုင်နားမှာ မသောင်း ပန်းဆိုင် ပေါ်လ

```
(base) ye@1st-gpu-server-197:~/ye/exp/sem-chunk/4github$
```

Fig. ma_thein.txt

ma_thein.chunk1.txt:

ရွှေတိဂုံဘုရားအ
နောက်ဘက်စောင်းတန်းမှာမ_သိန်းဆိုတဲ့ပန်းသည်တ_ယောက်ရှိတယ်။ပန်းသည်
မသိန်းပုံကိုသိပ္ပံမောင်_ဝရေးဖွဲ့ခဲ့ပုံက
...။
“မသိန်းသည်ပန်းသည်
တို့အနက်အတော်ပင်_ယဉ်စစရှိ၏။_အပြောလည်းကောင်း၊သ_ဘောလည်းကောင်း၊အပြု_အပြင်ကလည်းကောင်း၊_သူ့မှာအကောင်းချည်းစု_နေလေတော့သည်။သို့_ဖြစ်သောကြောင့်တ
ကျောင်းသားတွေသူ့ဆိုင်မှာ_ချည်းပန်းဝယ်ကြလေသည်။”_အဲ့ဒီခေတ်ကတက္ကသိုလ်
ကျောင်းသားတွေဟာစနေ_တနင်္ဂနွေနေ့တွေ
မှာအုပ်စုလိုက်ဘုရား_တက်ပြီးမသိန်းဆိုင်က_ပန်းတွေဝယ်၊မသိန်း
ဆိုင်ရှေ့ကခုံမှာထိုင်_ပြီးစကားတွေဖောင်ဖွဲ့_ကြပါသတဲ့။_တချို့ရက်တွေဆိုရှေ့_ကအဖွဲ့ဖောင်ဖွဲ့နေ_တုန်းနောက်တဖွဲ့ရောက်လာ_လို့မထချင်ထချင်_နဲ့နေရာဖယ်ပေးရ_ဆိုပဲ။ဒီထဲမောင်_သ်
ကတော့တကိုယ်တော်။_သူကမသိန်းဆိုင်သိမ်း_ချိန်အထိနေပြီးဆိုင်_သိမ်းပြီးရင်အတူပြန်_တဲ့ထိမသိန်းနဲ့ရင်း_နှီးတယ်။ဒီလိုနဲ့
ကျောင်းသားဟောင်းတွေကကျောင်း
ပြီးသွားလို့နေရပ်အ
သိသီးဆီဘဝခ
ရိုးချီတက်ကြ။_နောက်ကျောင်းသားအသစ်တွေ_ရောက်လာပြီးမသိန်းဆိုင်
မှာပန်းဝယ်ကြ။စ_ကားတွေဖောင်ဖွဲ့_ကြနဲ့_ဆယ်စုနှစ်နှစ်ခု_လောက်ရှိတော့မသိန်းဆိုင်_နားမှာမသောင်းပန်းဆိုင်_ပေါ်လာ။

Fig. ma_thein.chunk1.txt

Semantic Chunking Based on Cosine Distance of FastText Embeddings

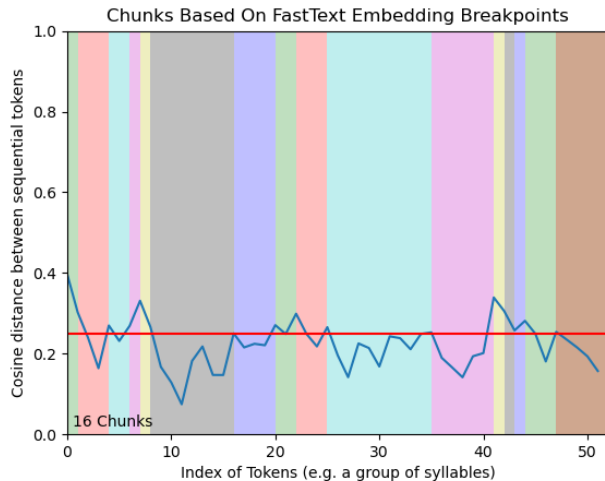


Fig. Semantic Chunking Based on Cosine Distance of FastText Embeddings

- Neural sequence labeling (BiLSTM-CRF etc.) improved on CRF-based segmentation by learning features instead of hand-crafting them
- Word embeddings (word2vec, fastText) opened the door to using **semantic similarity**, not just character patterns, as a segmentation/chunking signal
- NgaPi shows this idea can be pushed *above* the word level – semantic chunking for retrieval, not just word boundaries
- But there's a bigger shift coming: what if we stop trying to find “real words” at all, and instead let the **data** decide the vocabulary units? That's subword tokenization – next section

Why Subword Tokenization? The Motivation

- Neural MT/LLMs need a **fixed vocabulary** (a finite embedding matrix) – but language is open-ended: new words, names, typos, numbers, code
- Pure word-level vocabularies hit a hard wall: any word not in the fixed vocabulary becomes an <UNK> token – total information loss
- Pure character-level vocabularies avoid OOV entirely, but make sequences very long and harder for models to learn long-range structure from
- **Subword tokenization** is the middle ground: frequent words stay whole; rare/unseen words are broken into smaller, still-meaningful pieces
- Sennrich, Haddow & Birch (2016), *Neural Machine Translation of Rare Words with Subword Units* – the paper that brought **Byte-Pair Encoding (BPE)** into mainstream NLP

Byte-Pair Encoding (BPE): The Algorithm

- ① Start with a vocabulary of individual characters (or bytes)
- ② Count all adjacent symbol pairs in the training corpus
- ③ **Merge** the single most frequent pair into a new symbol; add it to the vocabulary
- ④ Repeat steps 2–3 for a fixed number of merges (this number sets the final vocabulary size)

Originally a **text compression algorithm** (Gage, 1994) – Sennrich et al. (2016) repurposed it for open-vocabulary NMT.

BPE: A Worked Mini-Example

Toy corpus: low low low lower lower newest newest widest

Merge #	Most frequent pair	New symbol
1	(e, s)	es
2	(es, t)	est
3	(l, o)	lo
4	(lo, w)	low
5	(n, e)	ne

After enough merges: low, lower = low+er, newest = ne+w+est, widest = w+i+d+est – frequent full words survive whole; rare words decompose into reusable, already-seen pieces.

- Schuster & Nakajima (2012); popularized by BERT (Devlin et al., 2019) and Google's NMT system
- Same incremental-merge structure as BPE, but the merge criterion is different: choose the pair that **maximizes training-data likelihood** under a unigram LM, not just raw frequency
- In practice: very similar output to BPE, but slightly better-motivated statistically
- Marks continuation pieces with ## (e.g. play, ##ing)

- Kudo (2018), *Subword Regularization* – a fundamentally different strategy from BPE/WordPiece
- Instead of **building up** a vocabulary via merges, it **starts with a huge candidate vocabulary** and **prunes it down** to a target size, keeping the subset that maximizes the likelihood of the training corpus (via EM)
- Key extra benefit: **multiple valid segmentations exist** for the same word, each with a probability – enables **subword regularization** (randomly sampling a segmentation each training step, like data augmentation) to make downstream models more robust

- Kudo & Richardson (2018) – a tokenizer/detokenizer library implementing **both BPE and Unigram LM**, designed to remove a major practical headache
- **Key innovation**: treats the input as a **raw character/byte stream**, including whitespace as an ordinary symbol (encoded as the special marker U+2581, `▯`) – **no language-specific pre-tokenization step required**
- This matters enormously for **Myanmar, Thai, Chinese, Japanese**: earlier subword methods assumed you already had word-segmented input to subword-ize further; SentencePiece removes that assumption entirely
- This is why SentencePiece (or similar byte/character-stream tokenizers) is the standard choice for training multilingual models today: mBART, mT5, XLM-R, and many open LLM tokenizers

- GPT-2 (Radford et al., 2019) introduced **Byte-level BPE (BBPE)**: run BPE merges over raw **UTF-8 bytes** (256 possible base symbols) instead of Unicode characters
- Guarantee: *any* input, in *any* language or script, or even non-text binary data, can always be represented – there is no “unknown character” possible even in principle
- Trade-off: for scripts with lots of multi-byte characters (Myanmar, Chinese, Thai, ...), a single character may cost **multiple byte-tokens** – directly related to the tokenizer-fairness issue in the next section
- This is the family of tokenizer used by GPT-2/3/4 and many subsequent LLMs (via OpenAI’s tiktoken library)

Comparing the Subword Methods

Method	Vocabulary Construction	Used By	Pre-tokenize?
BPE	Bottom-up frequency merges	GPT-2/3, RoBERTa	Usually yes
WordPiece	Bottom-up likelihood merges	BERT, Google NMT	Usually yes
Unigram LM	Top-down pruning + EM	ALBERT, mT5, XLNet	Usually yes
SentencePiece	BPE <i>or</i> Unigram, raw stream	mBART, mT5, LLaMA	No
Byte-level BPE	Bottom-up merges over bytes	GPT-2/3/4 (tiktoken)	No

- Solve the **open-vocabulary problem** without exploding sequence length like pure characters do
- Handle **morphologically rich languages** (agglutinative, compounding) gracefully – rare inflections decompose into a stem + known affixes
- Handle **numbers, code, URLs, and mixed-script text** in one unified scheme, without special-casing every category
- Enable **one shared vocabulary across many languages** in a single multilingual model – essential for today's LLMs
- But they are **not linguistically neutral**: a vocabulary trained mostly on English/web text will represent English efficiently and other languages/scripts far less efficiently – this is exactly the fairness problem we look at next

- GPT-family models use OpenAI's **tiktoken** library with byte-level BPE vocabularies:
 - `cl100k_base` (~100K tokens) – GPT-3.5/GPT-4 generation
 - `o200k_base` (~200K tokens) – newer GPT-4o-class models, with better multilingual/code coverage
- Other major LLM families (LLaMA, Gemini, Claude, Mistral, Qwen, ...) each ship their **own** SentencePiece- or BPE-based vocabulary, typically 32K–256K tokens
- Crucially: **the tokenizer is fixed once training begins** – you cannot swap it out later without re-training the embedding layer (and usually much more)

- Ahia, Kumar, Gonen, Kasai, Mortensen, Smith & Tsvetkov (2023), *Do All Languages Cost the Same? Tokenization in the Era of Commercial Language Models*, EMNLP 2023
- Analyzed OpenAI's API cost/utility across **22 typologically diverse languages**
- **Finding:** because tokenizer vocabularies are trained on data dominated by English/Latin-script text, speakers of many other languages are **systematically charged more tokens (and more money) for the same content** – while often getting **worse task performance**
- Related work: Petrov et al. (2023), *Language Model Tokenizers Introduce Unfairness Between Languages* – documents the same phenomenon across several tokenizer families
- This is a direct, measurable consequence of the **tokenization** choice made at pretraining time – not a downstream modeling issue

Why Does This Happen? Byte/Character Economics

- Byte-level BPE vocabularies allocate merged, multi-character “chunks” primarily to sequences that are **frequent in the training data**
- English/Latin-script text dominates most web-scraped pretraining corpora → common English words get **single-token** representations
- Scripts like Myanmar, Thai, Devanagari, or CJK characters are **multi-byte in UTF-8** and **underrepresented** in training data → they frequently fall back to **per-character or per-byte tokens**
- Net effect: the *same sentence meaning* can cost 2–10× more tokens in a low-resource language than in English, depending on the tokenizer
- Consequence chain: more tokens → higher API cost, higher latency, and **less usable context window** for the same amount of actual content

- In the LLM era, you usually **cannot choose your own tokenizer** – you must work within whatever vocabulary the pretrained model shipped with
- **Vocabulary extension**: adding new domain- or language-specific tokens (e.g. Myanmar syllables/words) to an existing pretrained tokenizer, then continuing pretraining to teach the model those new embeddings – an active area of research for under-served languages
- When **fine-tuning or building your own model from scratch**, you regain full control – and everything in this lecture (syllable vs. word vs. subword choice) becomes directly relevant again
- Budgeting: always estimate **tokens, not characters**, when planning LLM usage cost/latency for a given language – a Myanmar/Thai/Burmese-heavy application may need a very different budget than an English-only one

- An active research direction asks: can we **skip subword tokenization entirely** and model raw bytes/characters directly?
- **ByT5** (Xue et al., 2021) – a byte-level sequence-to-sequence Transformer, trading longer sequences for zero tokenizer bias
- **CANINE** (Clark et al., 2021) – a tokenization-free encoder operating directly on Unicode characters
- **MambaByte** and other state-space-model variants – byte-level modeling with sub-quadratic sequence handling, addressing the “longer sequence” cost of going byte-level
- Not yet mainstream for large commercial LLMs (compute cost of longer sequences is still a real barrier), but a promising direction for **fair, tokenizer-bias-free** multilingual modeling

- **Fertility**: average number of tokens produced per word (or per whitespace-unit) – lower is generally more efficient
- **Compression rate**: bytes of original text per token – higher means the tokenizer packs more information per token
- **Vocabulary coverage**: how rarely the tokenizer falls back to very short/byte-level pieces for a given language
- For classical word segmentation (syllable/word level), we still use **Precision, Recall, F-measure**, and boundary-level metrics like **EDWS** (Edit Distance of the Word Separator) – as used in my own COLING 2014 evaluation
- Good practice: always report **both** classical segmentation metrics (if you have gold-standard boundaries) *and* tokenizer fertility/compression (if evaluating for LLM use)

Segmentation Is the Foundation of Meaning Representation

- Every downstream vector representation – word2vec, fastText, contextual embeddings (BERT, GPT, ...) – is built **on top of** whatever units the tokenizer produced
- If segmentation is inconsistent or wrong, the resulting embedding space inherits that noise: the “same word” segmented two different ways gets **two different, unrelated vectors**
- This is why the choice of unit (character / syllable / word / subword) is not just an engineering detail – it directly shapes what a model *can represent* as a single semantic atom

- Bojanowski, Grave, Joulin & Mikolov (2016), *Enriching Word Vectors with Subword Information*
- Represents each word as the **sum of its character n-gram embeddings** – so morphologically related words (*play*, *playing*, *played*) automatically share sub-vectors
- Directly solves two problems at once: **OOV robustness** (an unseen word can still get a reasonable vector from its n-grams) and **morphology awareness** – both especially valuable for morphologically rich or under-resourced languages like Myanmar
- This is the embedding model underlying **NgaPi**'s semantic-similarity-based chunking, discussed earlier

- BERT, GPT, and virtually all modern LLMs compute a vector **per token**, where tokens are **subwords** (WordPiece/BPE/Unigram/SentencePiece), not whole words
- A “word-level” meaning is often reconstructed only *after* the model combines several subword-token vectors through attention/self-attention layers
- This means: **semantic composition happens above the tokenizer**, not at it – the tokenizer just decides the atomic units the model is allowed to compose from
- Practical consequence: comparing embeddings/similarity across two texts that were tokenized differently (e.g., different tokenizer vocabularies) is often **not directly comparable**

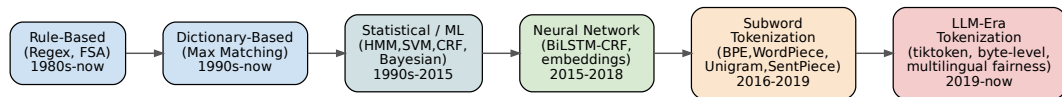
The Granularity Trade-off, Revisited

Granularity	Semantic Pro	Semantic Con
Character/Byte	Never OOV; universal	Very weak semantic atomicity per unit; long sequences
Syllable	Good middle ground for Myanmar/tonal languages; robust	Still below the level of a full “meaning unit” for many words
Word	Matches human intuition of “a unit of meaning”	OOV problem; sparse for rare/complex words
Subword (learned)	Balances vocabulary size and OOV robustness; data-driven	Splits can be linguistically arbitrary – not always aligned with real morphemes

There is no free lunch – every level trades off representability, sparsity, and sequence length differently. Choose empirically, per task (recall the Myanmar/Chinese MT findings from earlier).

- NgaPi demonstrates the natural next question once you accept “segmentation → embeddings → meaning”: can we go **one level up** and segment *sequences of words* into semantically coherent chunks?
- This is directly useful for **RAG systems**: retrieving chunks that preserve topical coherence (rather than fixed-length windows) improves retrieval quality
- The same fundamental idea recurs at every level of this lecture: **find natural boundaries using whatever signal is available** – character-class rules, dictionary+frequency, statistical likelihood, learned embeddings, or embedding-similarity drops

The Full Journey, Revisited



Notice: this is not strictly a story of “replacement” – rule-based syllable segmentation (syllbreak) and dictionary/DAG hybrids (oppaWord) remain state-of-the-art *for their specific purpose* even in the LLM era.

My Group's Tools Across This Timeline

Tool	Era / Approach	Core Idea	Link
sylbreak	Rule-based (2014)	One-line regex for syllable segmentation	github.com/ye-kyaw-thu/sylbreak
sylbreak4all	Rule-based (2021)	Extends regex approach to 9 Myanmar ethnic languages	(this paper, iSAL-NLP 2021)
Dict.+Unsup. Bayesian myWord	Statistical (2014)	Dirichlet Process + dictionary/LM augmentation	aclanthology.org/W14-5503
oppaWord	Dictionary + Viterbi (2021)	Unigram/bigram dict. + Viterbi decoding + NPMI phrases	github.com/ye-kyaw-thu/myWord
	Hybrid DAG+Bi-MM+LM (2025)	Fast, training-free hybrid segmenter	github.com/ye-kyaw-thu/oppaWord
NgaPi	Neural/embeddings (2024)	fastText-based semantic chunking	github.com/ye-kyaw-thu/NgaPi

Comparing All Approaches at a Glance

Approach	Data Needed	Strength	Weakness	Example
Rule-based	None	Fast, interpretable	Brittle, no generalization	sylbreak
Dictionary + MM	Lexicon only	Simple, decent recall	OOV kills precision	Thet et al. 2008
Dictionary + Viterbi	Lexicon + n-gram counts	Principled scoring	Still dictionary-bound	myWord
HMM / SVM / CRF	Labeled BIES corpus	Learns from data, strong accuracy	Needs annotated data	CRF Chinese seg.
Unsupervised Bayesian	Raw text only	No labels needed	Lower precision	Goldwater et al. 2009
Dict.+Bayesian hybrid	Raw text + dictionary	Best of both worlds	More complex to tune	Thu et al. 2014
Neural (BiLSTM-CRF)	Labeled corpus	Learned features, high accuracy	Compute-heavy	Chen et al. 2015
Hybrid DAG+Bi-MM+LM	Dictionary (+ optional LM)	Very fast, no training	Still dictionary-bound	oppaWord
Subword (BPE etc.)	Large raw corpus	Open-vocabulary, LLM-ready	Not linguistically "real" units	GPT/BERT tokenizers
Embedding-based chunking	Pretrained embeddings	Captures meaning, not just form	Needs good embeddings	NgaPi

- Most LLM tokenizers are still trained on corpora heavily skewed toward English/high-resource languages – **Myanmar and similar languages remain structurally disadvantaged** in token cost and representation quality
- Building large, clean, **manually word-segmented corpora** for low-resource languages remains a bottleneck (myWord's 12M-word corpus took years of collaborative annotation)
- Evaluating segmentation/tokenization quality for languages with **no agreed-upon gold standard** (e.g., what counts as “a word” in Myanmar is itself debated among linguists)
- Balancing **speed vs. accuracy vs. training-data requirements** – as oppaWord shows, a well-engineered classical hybrid can still beat neural approaches on real deployment constraints

- **Fair multilingual tokenizers:** research explicitly optimizing vocabulary allocation for equitable token cost across languages (morpheme-based encoding, balanced multilingual training corpora)
- **Tokenizer-free / byte-level models** (ByT5, CANINE, MambaByte): removing tokenizer bias entirely, at the cost of longer sequences
- **Vocabulary expansion / adaptation:** extending an existing pretrained LLM's tokenizer with new language- or domain-specific tokens without full retraining
- **Semantic / embedding-based chunking** (as in NgaPi) for RAG and long-document applications, moving beyond fixed-size or purely syntactic splitting
- **Hybrid classical+neural systems** (as in oppaWord) remaining practically important where speed, determinism, or lack of training data rule out large neural models
- Continued need for **low-resource language corpora and evaluation benchmarks** – the bottleneck is often data, not algorithms

- 1 Tokenization is not a solved, boring preprocessing step – it is a live research area with direct consequences for accuracy, cost, latency, and fairness
- 2 The “best” granularity (character/syllable/word/subword) is an **empirical, task- and language-dependent** question, not a fixed rule
- 3 Every historical paradigm (rules, dictionaries, statistics, neural nets, subwords) still has a place – newer is not always better for a given deployment constraint
- 4 In the LLM era, you often **inherit** a tokenizer rather than choosing one – understand its biases, especially for non-English/low-resource languages
- 5 Segmentation quality is the **foundation** all downstream vector representations and model capabilities are built on

References (1/5) – Foundations & Classical Methods

- ① Goldwater, S., Griffiths, T.L., Johnson, M. (2009). A Bayesian framework for word segmentation: Exploring the effects of context. *Cognition*, 112(1), 21–54.
- ② Goldwater, S., Griffiths, T.L., Johnson, M. (2006). Contextual dependencies in unsupervised word segmentation. *ACL-44*.
- ③ Papageorgiou, C.P. (1994). Japanese word segmentation by hidden Markov model. *Proc. HLT '94*.
- ④ Kudo, T., Matsumoto, Y. (2001). Chunking with support vector machines. *NAACL*.
- ⑤ Xue, N. (2003). Chinese word segmentation as character tagging. *Computational Linguistics and Chinese Language Processing*, 8(1).
- ⑥ Peng, F., Feng, F., McCallum, A. (2004). Chinese segmentation and new word detection using conditional random fields. *COLING*.
- ⑦ Tseng, H., Chang, P., Andrew, G., Jurafsky, D., Manning, C. (2005). A conditional random field word segmenter for SIGHAN bakeoff 2005.
- ⑧ Lafferty, J., McCallum, A., Pereira, F. (2001). Conditional random fields: Probabilistic models for segmenting and labeling sequence data. *ICML*.
- ⑨ Zhang, S., Mao, C., Yu, Z., Wang, H., Li, Z., Zhang, J. (2019). Word segmentation for Burmese based on dual-layer CRFs. *ACM TALLIP*, 18.
- ⑩ Htay, H.H., Murthy, K.N. (2008). Myanmar word segmentation using syllable level longest matching. *IJCNLP*, 41–48.

References (2/5) – Neural Segmentation & Embeddings

- 11 Chen, X., Qiu, X., Zhu, C., Liu, P., Huang, X. (2015). Long short-term memory neural networks for Chinese word segmentation. *EMNLP*.
- 12 Cai, D., Zhao, H. (2016). Neural word segmentation learning for Chinese. *ACL*.
- 13 Mikolov, T., Chen, K., Corrado, G., Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv:1301.3781*.
- 14 Mikolov, T., Sutskever, I., Chen, K., Corrado, G., Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *NeurIPS*.
- 15 Bojanowski, P., Grave, E., Joulin, A., Mikolov, T. (2017). Enriching word vectors with subword information. *TACL*, 5, 135–146.
- 16 Joulin, A., Grave, E., Bojanowski, P., Mikolov, T. (2016). Bag of tricks for efficient text classification. *arXiv:1607.01759*.
- 17 Bouma, G. (2009). Normalized (pointwise) mutual information in collocation extraction. *Proc. GSCL*.
- 18 Wang, Y., Zhou, L., Zhang, J., Zong, C. (2017). Word, subword or character? An empirical study of granularity in Chinese-English NMT. *arXiv:1711.04457*.
- 19 Viterbi, A. (1967). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Trans. Information Theory*, 13(2), 260–269.

- 20 Sennrich, R., Haddow, B., Birch, A. (2016). Neural machine translation of rare words with subword units. *ACL*, 1715–1725.
- 21 Schuster, M., Nakajima, K. (2012). Japanese and Korean voice search. *ICASSP*.
- 22 Devlin, J., Chang, M.W., Lee, K., Toutanova, K. (2019). BERT: Pre-training of deep bidirectional Transformers for language understanding. *NAACL*.
- 23 Kudo, T. (2018). Subword regularization: Improving neural network translation models with multiple subword candidates. *ACL*, 66–75.
- 24 Kudo, T., Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *EMNLP: System Demonstrations*, 66–71.
- 25 Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I. (2019). Language models are unsupervised multitask learners (GPT-2). OpenAI.
- 26 Gage, P. (1994). A new algorithm for data compression. *The C Users Journal*, 12(2).
- 27 Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I. (2017). Attention is all you need. *NeurIPS*.

- 28 Ahia, O., Kumar, S., Gonen, H., Kasai, J., Mortensen, D.R., Smith, N.A., Tsvetkov, Y. (2023). Do all languages cost the same? Tokenization in the era of commercial language models. *EMNLP*, 9904–9923.
- 29 Petrov, A., Malfa, E.L., Torr, P., Bibi, A. (2023). Language model tokenizers introduce unfairness between languages. *NeurIPS*.
- 30 Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., Raffel, C. (2022). ByT5: Towards a token-free future with pre-trained byte-to-byte models. *TACL*, 10.
- 31 Clark, J.H., Garrette, D., Turc, I., Wieting, J. (2022). CANINE: Pre-training an efficient tokenization-free encoder for language representation. *TACL*, 10.
- 32 Kamradt, G. The 5 levels of text splitting for retrieval. (video reference used in NgaPi documentation).

References (5/5) – Myanmar Language NLP (Author's Own Work)

- 33 Thu, Y.K., Finch, A., Sumita, E., Sagisaka, Y. (2014). Integrating dictionaries into an unsupervised model for Myanmar word segmentation. *Proc. 5th Workshop on South and Southeast Asian NLP, COLING 2014*, 20–27. <https://aclanthology.org/W14-5503/>
- 34 Thu, Y.K., Myat Nwe, H., Aye Thant, H., Man Htun, H., Mon, H., Myat Khaing, M.M., Pan Oo, H., Phyu, P., Aeindray Kyaw, N., Myint Oo, T., Oo, T., Zin, T.T., Oo, T. (2021). sylbreak4all: Regular expressions for syllable breaking of nine major ethnic languages of Myanmar. *iSAI-NLP 2021*.
- 35 Thu, Y.K. (2014). sylbreak: Syllable segmentation tool for Myanmar language. <https://github.com/ye-kyaw-thu/sylbreak>
- 36 Thu, Y.K. (2021). myWord: Syllable, word and phrase segmenter for Burmese. <https://github.com/ye-kyaw-thu/myWord>
- 37 Thu, Y.K. (2025). oppaWord: Hybrid DAG+Bi-MM+LM Myanmar word segmenter. <https://github.com/ye-kyaw-thu/oppaWord>
- 38 Thu, Y.K. (2024). NgaPi: Semantic chunker for Burmese based on fastText embeddings. <https://github.com/ye-kyaw-thu/NgaPi>
- 39 Thu, Y.K., Finch, A., Sagisaka, Y., Sumita, E. (2013a). A study of Myanmar word segmentation schemes for statistical machine translation. *Proc. ICCA 2013*, 167–179.
- 40 Ding, C., Thu, Y.K., Utiyama, M., Sumita, E. (2016). Word segmentation for Burmese (Myanmar). *ACM TALLIP*, 15(4), 22:1–22:10.
- 41 Thet, T.T., Na, J.C., Ko Ko, W. (2008). Word segmentation for the Myanmar language. *Journal of Information Science*, 34(5), 688–704.

Compiled by Dr. Ye Kyaw Thu, July 12, 2026.

Thank You! Questions?

Dr. Ye Kyaw Thu

yktnlp@gmail.com

<https://sites.google.com/site/yekyawthunlp/>